

製造指示 登録・編集 — 画面処理設計

001_DD-SPD — 001_DD を詳細化し、001_BD（SCR-001）を実装する。要件 REQ-010～REQ-019。

改版履歴

項目	値
ドキュメントID	001_DD-SPD
システム名	ProductionManagementAI
サブシステム名	製造指示（Production orders）
作業項目	WI-002
作成者	Claude（ThanhTN 向け）
作成日	2026-09-18
最終更新者	Claude（ThanhTN 向け）
最終更新日	2026-09-22

版	日付	作成者	改版内容
1	2026-09-18	Claude （ThanhTN 向け）	新規作成。001_DD リビジョン 1 の処理フロー P-01～P-05 を本書に移し、コンポーネントごとに分ける
2	2026-09-22	Claude （ThanhTN 向け）	WI-004（DEC-016）：ページは共通の <code>AppHeader</code> の中に描画され、そこにナビゲーションバー（ <code>AppNavbar</code> 、003_DD モジュール 10 と 003_DD-SPD §8 で設計）が入る。パンくずリストのリンクは <code>GuardedLink</code> になる（003_DD モジュール 11）。本画面の処理ステップの変更なし。WI-004 DEC-022：フォームは変更の有無を共通の <code>NavigationGuard</code> （003_DD モジュール 11、003_DD-SPD §9）に登録するので、ナビゲーションバー、パンくずリスト、アプリ名のリンクは、保留中の遷移先付きで既存の破棄ダイアログを開く（001_BD v7 E-07a）。破棄を選ぶとそこへ遷移する
3	2026-09-23	Claude （ThanhTN 向け）	WI-005：UI を日本語化。ページタイトル、ボタン名、ダイアログのタイトルを日本語のテキストで示す（WI-005 DEC-003）。項目・ルール・API・処理の変更はない

概要と処理の一覧

項目	値
画面／ファイル名	src/frontend/src/features/production-orders/ProductionOrderPage.tsx とそれが組み立てるコンポーネント。各リクエストのサーバー側は ProductionOrdersController にある
概要	SCR-001 がどう読み込み、検証し、保存し、キャンセルし、各 API の結果に反応するか。コンポーネントごとに 1 ブロック

No	処理名	概要	備考
1	ProductionOrderPage — P-01 読み込み	ロールのゲート、並列の取得、ページの状態、タイトル、フラッシュメッセージ	E-01
2	ProductionOrderForm — P-02 登録 / P-03 更新（クライアント側）	フォーカス喪失時／送信時の検証、リクエストの組み立て、レスポンスの処理	E-02～E-06、E-08
3	ProductionOrderForm — P-05 保存レスポンスの処理	各 HTTP の結果の UI への対応付け	E-06
4	DiscardChangesDialog — P-04 キャンセル	変更の有無のチェックと確認	E-07、E-09
5	ProductionOrdersController / ProductsController — リクエストの境界	認証、コンテンツタイプ、サービスへの振り分け、結果 → HTTP	サーバーのステップは 001_DD-FN にある

参考文献

No	文書	目的／用途	備考
1	001_DD	項目、状態、メッセージカタログ モジュールの一覧	親文書
2	001_DD-API	下で呼ぶエンドポイントの仕様	
3	001_DD-FN	サーバー側のステップ（ CreateAsync 、 UpdateAsync 、 ... ）	
4	001_BD	イベント E-01～E-09、バリデーション V-01～V-08	

処理の設計

1. ProductionOrderPage — P-01 読み込み

項目	値
詳細	/production-orders/new と /production-orders/:id のルートコンポーネント
作成者／日付	Claude / 2026-09-18

項目	値
最終更新者／日付	—

処理概要：ルートからモードを決め、クライアント側のロールのゲートを適用し、データを取得してから、フォームかページ全体のパネルを描画する。マウント時と **再読み込み／もう一度試す** で実行する。

使用するコンポーネント／サービス

No	名前	概要	備考
1	useAuth	現在のユーザーとロール	WI-001
2	api.ts （ listProducts 、 getOrder ）	apiClient の型付きラッパー	001_DD モジュール 10
3	ProductionOrderForm 、 MessageBanner	準備完了で描画	
4	useLocation / useNavigate	登録後のリダイレクトからのフラッシュメッセージ	react-router

処理フロー

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
1	ルートが /new ならモード= create 、それ以外は id 付きの edit	—	—	—
2	user.roles でロールのゲート	Admin も Operator もない → 終了	—	forbidden パネル（MSG-E012）、API 呼び出しなし
3	状態を loading にし、document.title をそのモードのタイトルにする（指示番号は読み込み後に加える）	—	—	スケルトンのカード、aria-busy="true"
4	並列に取得：製品と、編集モードでは製造指示	—	GET /api/products 、 GET /api/production-orders/{id}	—
5	製造指示のレスポンス	404 → notFound 。401 → apiClient が /login へリダイレクト。403 → forbidden 。ネットワーク／5xx → loadError	apiClient	見つからない（MSG-E011）／権限なし／読み込みエラー（MSG-E013 + もう一度試す）のパネル。フォーカスはパネルへ

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
6	製品のレスポンス	失敗 → <code>loadError</code> 。空のリスト → 選択枝なしで続行	<code>apiClient</code>	製品のセレクトは MSG-E014 を表示して無効
7	<code>initialValues</code> を組み立てる：製造指示から（編集）、またはステータス <code>Draft</code> の空（登録）。編集モードではタイトルを <code>製造指示 {orderNumber}</code> – <code>ProductionManagementAI</code> （登録モードは <code>新規製造指示 – ProductionManagementAI</code> ）にする	—	—	<code>ready</code>
8	<code>location.state.flash</code> が設定されていれば（登録後）そのメッセージを表示し、 <code>navigate(location.pathname, { replace: true, state: null })</code> で状態をクリア	フラッシュなし → スキップ	—	成功バナー MSG-I001

2. ProductionOrderForm — P-02 登録 / P-03 更新（クライアント側）

項目	値
詳細	項目 6～16 の制御フォーム。値、エラー、変更の有無、保存のリクエストを持つ
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

処理概要：早期のフィードバックのためにフォーカス喪失時に検証し、保存時にすべてを検証してから 1 回リクエストを送る。サーバーはすべてのチェックを繰り返すので（001_DD-FN）、クライアントのチェックは参考である。更新に成功すると、フォームはサーバーのレスポンスを新しい基準にするので、`isDirty` は `false` になり、ロックとステータスの選択枝は新しいステータスを反映する。

使用するコンポーネント／サービス

No	名前	概要	備考
1	<code>validation.ts</code>	<code>validateProduct</code> 、 <code>validateQuantity</code> 、 <code>validateDueDate(value, initial, mode)</code> 、 <code>validateNotes</code> → メッセージ ID または <code>null</code>	V-01～V-05 を再現
2	<code>messages.ts</code>	メッセージ ID → テキスト	001_DD のカタログ
3	<code>api.ts</code> （ <code>createOrder</code> 、 <code>updateOrder</code> ）	POST/PUT のラッパー	001_DD-API §2、§4

No	名前	概要	備考
4	FieldError、MessageBanner	エラーの表示	

処理フロー — フォーカス喪失（E-02）とステータスの変更（E-08）

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
1	項目からフォーカスが外れる	読み取り専用の項目 → スキップ	—	—
2	その項目のバリデーターを実行	納期：「ブラウザのローカルの今日以降」は登録モードか、値が <code>initialValues.dueDate</code> と異なるときだけチェック（DEC-009、DEC-021）	<code>validation.ts</code>	<code>aria-invalid</code> 付きのインラインのエラー、またはエラーをクリア
3	備考の入力（E-03）	—	—	文字数 <code>{n}/500</code> （コードポイント）。500 超で赤
4	ステータスの変更	選択肢は <code>[current, ...allowedNextStatuses]</code> のみ	—	値はローカルで変わる。保存するまで何も保存しない

処理フロー — 保存（E-04）

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
1	すべてのバリデーターを実行（読み取り専用の項目はスキップ）	エラーあり → すべて表示し、DOM の順で最初の不正な項目にフォーカスして終了	<code>validation.ts</code>	バリデーションエラーの状態
2	ボディを組み立てる： <code>productId</code> 、 <code>quantity</code> （数値として）、 <code>dueDate</code> 、 <code>notes</code> （前後の空白を除去、 <code>""</code> → <code>null</code> ）。編集モードでは <code>initialValues</code> から <code>status</code> と <code>version</code> も	—	—	—
3	<code>saving</code> にする：保存ボタンを無効にし、ラベルを「保存中...」にする	—	—	保存中の状態
4	送信	登録 → POST、編集 → PUT	<code>createOrder</code> / <code>updateOrder</code>	—

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
5	登録の成功（201）	—	<code>navigate('/production-orders/{id}', { replace: true, state: { flash: 'MSG-I001' } })</code>	ページが編集モードで P-01 を再実行。replace により、戻るで空の登録フォームに戻らない
6	更新の成功（200）	—	—	<code>initialValues = values = response</code> 。バナー MSG-I002（role="status"）。ロックとステータスの選択肢を <code>isProductQuantityEditable / allowedNextStatuses</code> から再計算
7	失敗	—	処理 3（P-05）	—
8	<code>saving</code> をクリア	常に	—	保存ボタンを再び有効に

3. ProductionOrderForm — P-05 保存レスポンスの処理（E-06）

項目	値
詳細	成功以外のすべての保存の結果を UI に対応付ける
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

処理概要：apiClient からの `ApiError` はステータスと解析した Problem Details を持つ。入力値は、401（セッション切れ）の後か、409 の後にユーザーが **再読み込み** を押した場合を除き、常に保持する。

使用するコンポーネント／サービス

No	名前	概要	備考
1	<code>apiClient</code> の <code>ApiError</code>	<code>{ status, problem? }</code>	001_DD モジュール 10
2	<code>messages.ts</code>	ID → テキスト	

処理フロー

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
1	400 <code>VALIDATION</code>	<code>problem.errors</code> の各キーについて → その項目のメッセージ	—	インラインのエラー、最初の不正な項目にフォーカス
2	401	—	<code>apiClient</code> の <code>onUnauthorized</code>	認証をクリア → <code>ProtectedRoute</code> → <code>/login</code> （未保存の値は失われる。許容）
3	403	—	—	権限なしのパネル（MSG-E012）
4	404 <code>MSG-E011</code>	—	—	見つからないパネル
5	409 <code>MSG-E009</code>	—	—	再読み込み 付きのエラーバナー → P-01（ローカルの編集を破棄）
6	422 <code>MSG-E007</code> / <code>MSG-E008</code>	—	—	そのメッセージのエラーバナー
7	415、5xx、ネットワークエラー	—	—	エラーバナー <code>MSG-E013</code> 。保存ボタンを再び有効に

4. DiscardChangesDialog — P-04 キャンセル（E-07、E-09）

項目	値
詳細	未保存の変更を破棄する前の確認（REQ-019、DEC-018）
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

処理概要：キャンセルは `isDirty`（値を `initialValues` と比較、備考は前後の空白を除去）をチェックする。変更のあるフォームだけがダイアログを開く。対象はキャンセルボタンだけで、ブラウザの戻るやタブを閉じる操作は対象外（DEC-018）。

使用するコンポーネント／サービス

No	名前	概要	備考
1	ネイティブの <code><dialog></code> + <code>showModal()</code>	フォーカストラップ、Escape、 <code>:::backdrop</code>	ライブラリなし
2	<code>useNavigate</code>	<code>/production-orders</code> へ離れる	WI-003 以降は SCR-002

処理フロー

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
1	キャンセルをクリック	<code>isDirty</code> が <code>false</code> → <code>navigate('/')</code>	—	ホーム
2	ダイアログを開く	<code>isDirty</code> が <code>true</code>	<code>dialog.showModal()</code>	ダイアログ。フォーカスは 編集を続ける
3	破棄	—	<code>dialog.close()</code> 、 <code>navigate('/')</code>	ホーム。何も保存しない
4	編集を続ける または <code>Escape</code> （ <code>cancel</code> イベント）	—	<code>dialog.close()</code>	すべての値を保ったフォーム。フォーカスはキャンセルに戻る

5. ProductionOrdersController / ProductsController — リクエストの境界

項目	値
詳細	001_DD-API の HTTP の入口。業務ロジックなし
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

処理概要：ASPNET Core のパイプラインが認証と認可を行い、次にコンテンツタイプをチェックする。アクションはサービスのメソッドを 1 つ呼び、その `Result` を HTTP に対応付ける。業務のチェックはすべて 001_DD-FN で行う。

使用するコンポーネント／サービス

No	名前	概要	備考
1	ポリシー <code>ProductionOrderEditor</code>	<code>RequireRole("Admin", "Operator")</code>	DEC-001
2	<code>[Consumes("application/json")]</code>	他のコンテンツタイプでは 415	DEC-020
3	<code>ProductionOrderService</code>	ユースケース	001_DD-FN
4	<code>ProblemResults</code> ヘルパー	<code>Result</code> → Problem Details（ <code>type</code> 、 <code>title</code> 、 <code>status</code> 、 <code>code</code> 、 <code>errors</code> 、 <code>traceId</code> ）	DEC-023

処理フロー

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
1	認証（Cookie）	なし → 401、空のボディ	ASPNET Core	—
2	認可（ポリシー）	ルールなし → 403、空のボディ	ASPNET Core	—
3	ルート的一致 <code>{id:guid}</code>	GUID でない → 404	ルーティング	—

ステップ	説明	分岐／条件	呼び出し	結果（状態／リダイレクト／描画）
4	コンテンツタイプ（POST/PUT）	JSON でない → 415	[Consumes]	—
5	モデルバインド	バインドのエラー → メッセージ ID 付きの 400（独自の <code>InvalidModelStateResponseFactory</code> ）	MVC	—
6	サービスのメソッドを呼び出す	—	ListProductsAsync / GetAsync / CreateAsync / UpdateAsync	—
7	結果を対応付ける	Ok → 200（登録では 201 + Location）、Invalid → 400、NotFound → 404、Conflict → 409、RuleViolation → 422	ProblemResults	レスポンス
8	未処理の例外	—	AddProblemDetails + 例外ハンドラー	500 MSG-E013、例外の詳細なし

未解決の決定

なし。関連する決定： `work-items/WI-002/decisions.md` の DEC-007、DEC-008、DEC-009、DEC-018、DEC-020、DEC-021、DEC-023。